

# Object–Oriented Programming

## Lecture 4: OOP – 1 (Part2)

**Prof. Jaehyung Seo**

(seojae777@konkuk.ac.kr)

**Language Intelligence & Representation Lab.**

# Expecting the Unexpected

**Central Theme:** "Handling the Unforeseen"

## Traditional Approach

- Return error codes / Check conditions before acting

## Pythonic Approach

- Exceptions: Raise → Catch → Recover / Clean up

## Connection to Inheritance

- Exceptions are objects — inherit from **BaseException**
- IS-A: **ValueError** IS-A **Exception** IS-A **BaseException**
- try/except uses **polymorphism** to match handlers

*"Easier to Ask Forgiveness than Permission"*

# What is an Exception?

## Definition

- An exception is a special **error object** raised when a normal response is impossible
- Not just a message — has **.args**, methods, and state

## How It Behaves

- Alters the normal **flow of control** — execution **stops immediately**
- Propagates up the **call stack** until caught or program terminates
- $\neq$  return: return goes to caller; raise **skips through** multiple callers

# Triggering Built-in Exceptions

## Code Snippet:

```
# Python automatically raises exceptions for invalid operations
def calculate_ratio(x: int, y: int) -> float:
    return x / y  # Raises ZeroDivisionError if y == 0

# You can also manually trigger existing exceptions
def check_type(value: any) -> None:
    if not isinstance(value, int):
        raise TypeError("Only integers are allowed!")
```

# The "Raise" Mechanism

## Analogy: A Referee Calling a Foul

- When code detects an illegal state, it "raises" an exception (blows the whistle).

## How It Works

- The **raise** keyword creates an exception instance and throws it.
- Normal execution **stops immediately** at the raise statement.
- Control transfers to the nearest exception handler on the call stack.

# Custom Validation with raise

## Code Snippet:

```
class EvenOnly(list): # inherit from built-in list, not typing.List
    def append(self, value: int) -> None:
        if not isinstance(value, int):
            raise TypeError(
                "Only integers can be added")
        if value % 2 != 0:
            raise ValueError(
                "Only even numbers can be added")
        super().append(value)
```

# The Call Stack Unwinding

## What Happens When an Exception is Raised?

1. **Trigger**: raise Exception is executed.
2. **Halt**: The current function stops executing further lines.
3. **Bubble Up**: Control passes back to the calling function.
4. **Crash**: If no handler catches it, Python prints a Traceback and exits.

# Unreachable Code

## Code Snippet:

```
def never_returns() -> NoReturn:
    print("I am about to raise an exception")
    raise Exception("This is always raised")
    print("This line will never execute")

def call_excepter() -> None:
    print("Call starts...")
    never_returns()
    print("Call ends...")    # Unreachable!
```



# Recovering from Errors

## Before (Unhandled)

- `never_returns()` → Exception bubbles to Interpreter → **Program Crashes**

## After (Handled with `try...except`)

- `never_returns()` → Exception caught by `except block` → **Program Continues**

## Key Insight

- The `try...except` construct lets your program survive errors gracefully.

# The try...except Syntax

## Code Snippet:

```
def funny_division(divisor: int) -> float:
    try:
        if divisor == 13:
            raise ValueError("13 is unlucky")
        return 100 / divisor
    except ZeroDivisionError:
        return "Zero is not a good idea!"
    except TypeError as e:
        print(f"Caught a type error: {e}")
        return "Enter a numerical value"
```

```
#funny_division(5) #funny_division(0) #funny_division("hello") #funny_division(13)
```

# The Complete try Block

## Complete Syntax

```
try:
    result = 100 / divisor

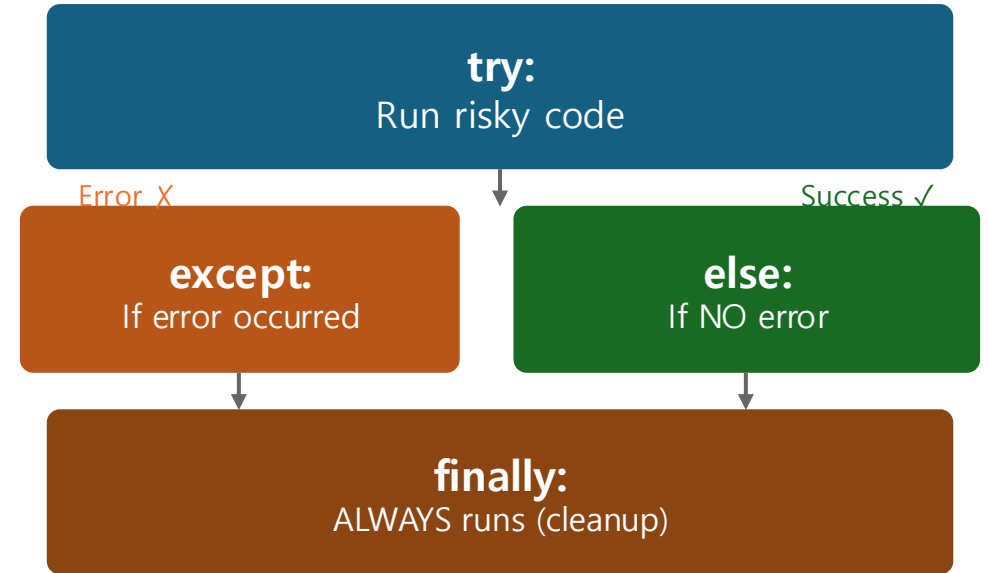
except ZeroDivisionError:
    print("Cannot divide by zero")

except TypeError as e:
    print(f"Wrong type: {e}")

else:
    print(f"Success: {result}")

finally:
    print("Cleanup done")
```

## Execution Flow



### Key Rules:

- else runs **only if no exception**
- finally **always runs** (even after return)
- Use finally for **resource cleanup** (files, DB)

# Python's Exception Hierarchy

## Tree Structure:

**BaseException** (Top)

├── SystemExit, KeyboardInterrupt

└── **Exception** (Main branch)

    ├── ArithmeticError → **ZeroDivisionError**

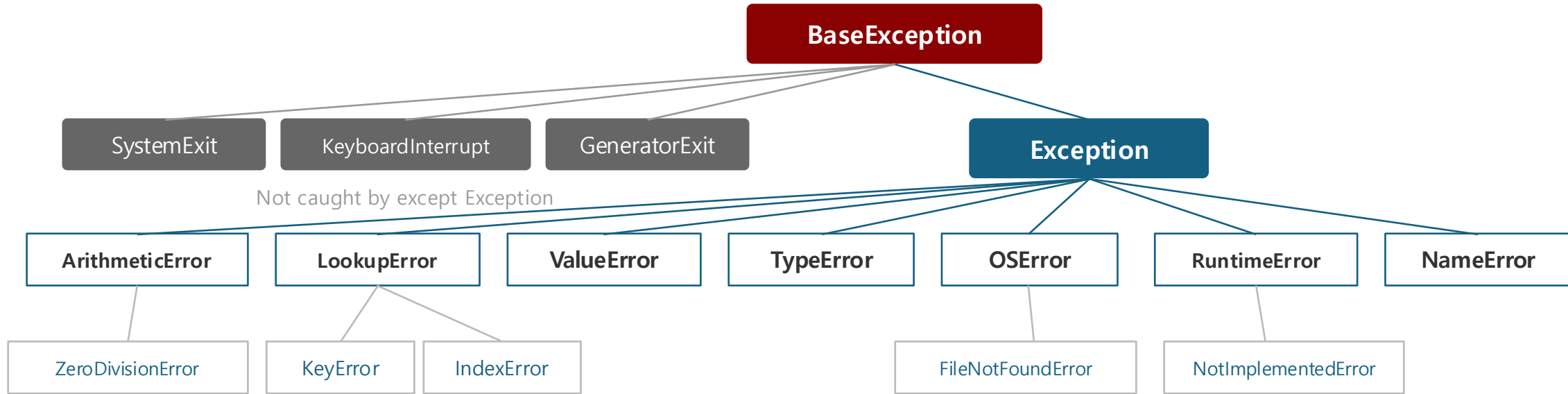
    ├── LookupError → **KeyError, IndexError**

    ├── **ValueError, TypeError**

    └── OSError → FileNotFoundError

- Always catch **Exception**, never **BaseException**

# Exception Hierarchy — Visualized



## Best Practice

- Catch **Exception**, never BaseException
- Be as **specific** as possible (catch ValueError, not Exception)

## Custom Exceptions

- Inherit from **Exception** (not BaseException)
- Create domain hierarchy (e.g., **AppError** → **ValidationError**)

■ Root (BaseException)   ■ Non-catchable   ■ Catchable (Exception branch)   □ Concrete exceptions

# Defining Custom Exceptions

## Key Takeaway

- If built-in exceptions lack the semantic meaning your application needs, create a **custom exception class**.

## How To

- Simply inherit from `Exception` (or a more specific subclass like `ValueError`).
- Override `__init__` to store extra context (amounts, balances, etc.).

## Benefits

- Callers can catch your **specific exception type** and access its attributes.

# Crafting Custom Exception Classes

## Code Snippet:

```
class InvalidWithdrawal(ValueError):  
    """Raised when withdrawal exceeds balance."""  
    def __init__(self, balance: float,  
                  amount: float):  
        super().__init__(  
            f"Has {balance},  
            tried {amount}")  
        self.balance = balance  
        self.amount = amount  
  
    @property  
    def overshoot(self):  
        return self.amount - self.balance
```

# EAFP vs LBYL

## LBYL: Look Before You Leap

- Check conditions before execution (if/else guards).
- Verbose; checks and actions are separate (two accesses).

## EAFP: Easier to Ask Forgiveness than Permission

- Just try the operation; catch exceptions if it fails.
- Clean, **Pythonic**, one access instead of two.  
(In file I/O or threading, also avoids TOCTOU race conditions.)

## Verdict

- Python community strongly prefers **EAFP**.



# Exceptions for Flow Control

## Code Snippet:

```
# LBYL: Look Before You Leap (Messy)
if item_name in inventory:
    if inventory[item_name] > 0:
        inventory[item_name] -= 1

# EAFP: Easier to Ask Forgiveness (Pythonic)
try:
    inventory[item_name] -= 1
except KeyError:
    raise InvalidItemType(item_name)
except ValueError:
    raise OutOfStock(item_name)
```

# Case Study – Data Validation

## The Goal

- Ensure inputs to our Iris k-NN classifier are completely valid.

## The Threat

- Files might have **missing data**, **non-numeric strings**, or **invalid species names**.

## The Strategy

- Use Python exceptions to **discover** exceptional data (raise) and **respond** to it (catch and report).

# System Context and Actors

## Actor 1: "Botanist"

- Provides Training Data → System checks validity.

## Actor 2: "Researcher"

- Submits Unknown Samples → System classifies.

## System Boundary: k-NN Classifier

- Load CSV → Validate → Create objects → Classify
- Exceptions handle invalid data at each step.

# Object Creation Architecture

## Hierarchy of Object Relationships:

**TrainingData** (loads CSV)

- creates → **KnownSample** (inherits Sample)
- has measurements: sepal\_length, sepal\_width, petal\_length, petal\_width
- has species (validated)

## Validation Points:

- float() conversion → may raise **ValueError**
- Species check → may raise **InvalidSampleError**

# Identifying Points of Failure

## The Pitfalls:

- Data values are strings that **cannot be converted** to float (e.g., "N/A").
- Species strings that don't match the **three known Iris types**.

## The Solution:

- Create an **InvalidSampleError(ValueError)** to cleanly separate data layout errors from bugs in Python math logic.

# Catching Bad Behavior

## Definition

- Exceptions enforce **access control** and **business rules** against illogical operations.

## Example

- Trying to classify **before** loading training data → Raise exception
- Running the test suite **before** loading data → Raise exception

## Pattern

- Guard methods check **preconditions** and raise if violated.

# The Data Parsing Pipeline

## How to Safely Instantiate Objects from Raw Strings:

1. Receive a dictionary representing one CSV row.
2. Verify the '**species**' matches expected values.
3. Apply `float()` to measurement fields inside a try block.
4. If `ValueError` occurs, use `raise ... from` to trigger `InvalidSampleError`.
5. Return a valid **Sample** object.

# The from\_dict Factory (Code)

## Code Snippet:

```
class InvalidSampleError(ValueError): pass

class Sample:
    @classmethod
    def from_dict(cls, row: dict) -> 'Sample':
        try:
            return cls(
                sepal_length=float(row["sepal_length"]),
                sepal_width=float(row["sepal_width"]),
                petal_length=float(row["petal_length"]),
                petal_width=float(row["petal_width"]))
        except ValueError as ex:
            raise InvalidSampleError(
                f"Invalid: {row!r}") from ex
```



# Why @classmethod?

## Regular method vs. classmethod

|                                      |                                    |
|--------------------------------------|------------------------------------|
| <code>def method(<b>self</b>)</code> | ← receives the <b>instance</b>     |
| <code>def method(<b>cls</b>)</code>  | ← receives the <b>class itself</b> |

## Why it matters for Inheritance

- `cls` = whatever class called the method
- `Sample.from_dict(row)` → `cls` is **Sample**
- `KnownSample.from_dict(row)` → `cls` is **KnownSample**

→ Write `from_dict` **once in the superclass**, subclasses inherit it for free.

# Validating Categorical Data

## Key Takeaway

- Validate sets of known categories (enumerations) before converting strings.

## Supporting Technique

- Define a Python **set** of valid species strings.
- Use the **in** operator for O(1) lookup.
- If not in the set → raise **InvalidSampleError** immediately.

## Benefit

- Fails fast and clearly before any expensive computation.

# Strict Set Validation (Code)

## Code Snippet:

```
VALID_SPECIES = {  
    "Iris-setosa",  
    "Iris-versicolor",  
    "Iris-virginica"  
}  
  
@classmethod  
def from_dict(cls, row: dict) -> 'KnownSample':  
    if row["species"] not in VALID_SPECIES:  
        raise InvalidSampleError(  
            f"Invalid species: {row['species']}")
```

# The Outer Processing Loop

## Code Snippet:

```
def load(self, raw_data_iter):
    for n, row in enumerate(raw_data_iter):
        try:
            # Validation + instantiation
            sample = KnownSample.from_dict(row)
            self.samples.append(sample)
        except InvalidSampleError as ex:
            # Graceful reporting, loop continues
            print(f"Row {n+1} skipped: {ex}")
```

# Recall

## Reflection Questions:

- Q1.** What does the **raise** statement do to the sequential execution flow?
- Q2.** How does Python's **exception hierarchy** help us design except clauses?
- Q3.** What does "**EAFP**" mean, and why is it preferred in Python over LBYL?

# Summary

## "Exceptions are a Powerful Communication Tool."

In this chapter, we learned that exceptions are not just for fatal crashes. They are an elegant mechanism for **validating inputs, routing control flow**, and cleanly separating expected logic from unexpected data.